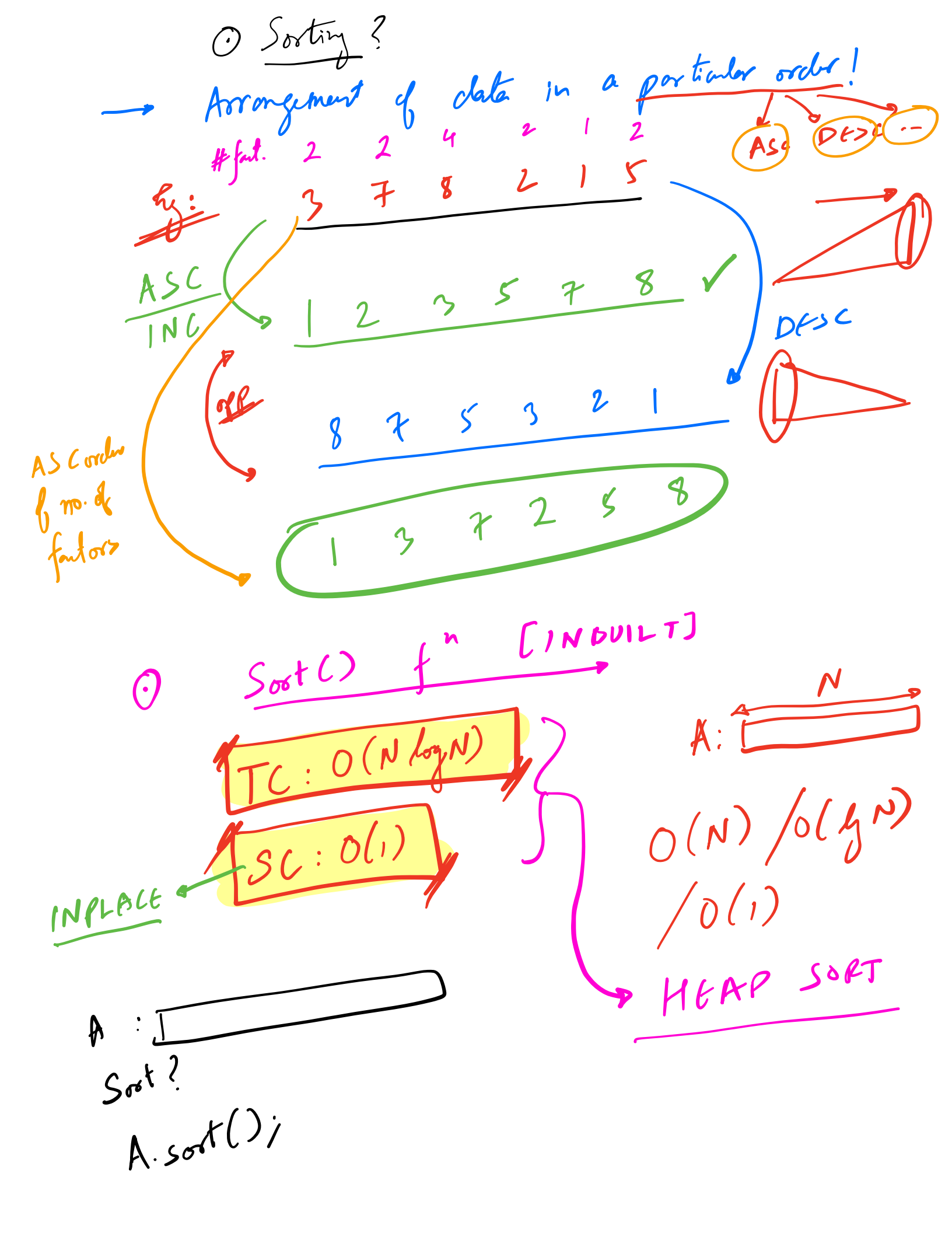
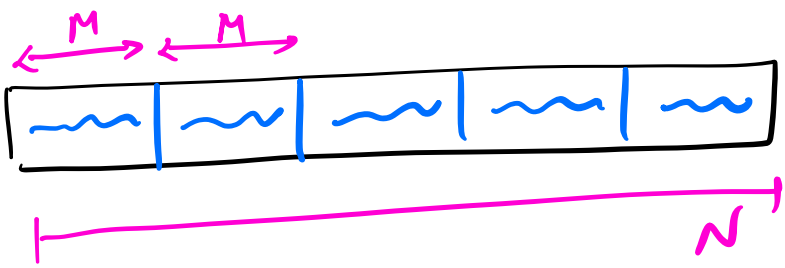
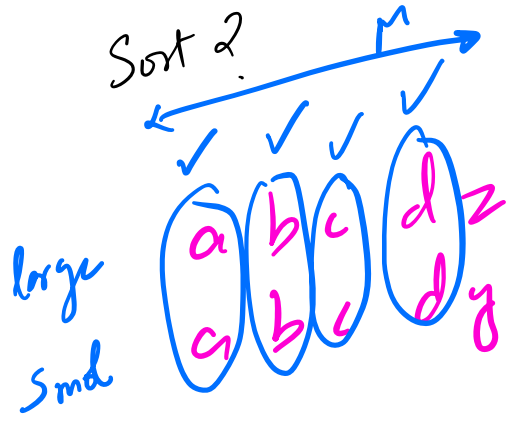

Sorting ?
Arrangement of data in a particular order !
ASC
DESC
--
#fact. 2 2 4 2 1 2
Eg: 3 7 8 2 1 5
ASC / INC
1 2 3 5 7 8
DESC
8 7 5 3 2 1
ASC order of no. of factors
1 3 7 2 5 8
Sort() f^n [INBUILT]
TC : O(N logN)
SC : O(1)
INPLACE
A:
N
O(N) / O(logN) / O(1)
HEAP SORT
A :
Sort ?
A.sort();

Array of strings

A :



Sort 2.



O(1) int vs int

O(M) str vs str

TC : O(M x N log N)

Q Given N elements, at every step remove an element!

Cost of removing an element : Sum of the elements present in the array before removing this element.

FIND the MIN cost to remove all elements

A : [2, 1, 4]

-2 [1, 4]

-1 [4]

-4 []

Cost
7
+ 5
+ 4
16

[4 6 1]	Cost
-6 [4, 1]	11
-4 [1]	+ 5
-1	+ 1
	17

A : [2, 1, 4]

		Cost
		7
-4	[2, 1]	+ 3
-2	[1]	+ 1
-1	[ ]	11

Observation

(a, b, c, d)

-a	Largest	(b, c, d)
-b		(c, d)
-c		(d)
-d	smallest	( )

Cost

$a + b + c + d$

$b + c + d$

$c + d$

d

$a + 2b + 3c + 4d$

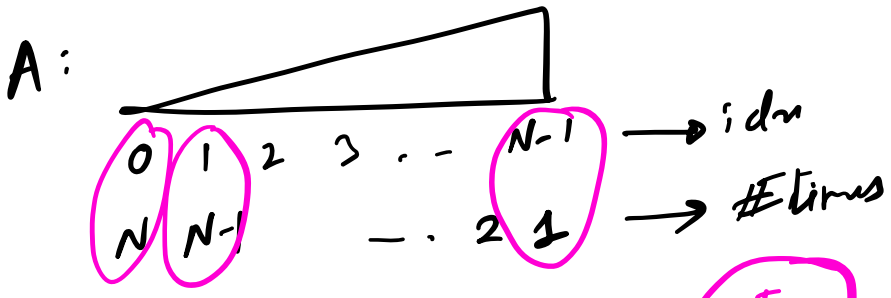
A : [2 1 4] → Sort ASC

idx 0 1 2

A : [1, 2, 4]

Contribution: ×3 + ×2 + ×1

$3 + 4 + 4 \rightarrow 11$



$i + \#times = N$

$\#times = N - i$

```
// A[]
A.sort(); //ASC
sum = 0;
for(i=0 → N-1){
    sum += A[i] x (N-i);
}
out sum;
```

NlgN + N

TC: O(NlgN)

SC: O(1)

[DISTICT]

Noble Integer

Given N array elements. Calculate the no. of Noble elements.

Noble element: $A[i]$ { No. of elements $< A[i]$ } $= A[i]$

A. [1, -5, 3, 5, -10, 4] → 3

1	-5	3	5	-10	4
2	1	3	5	0	4

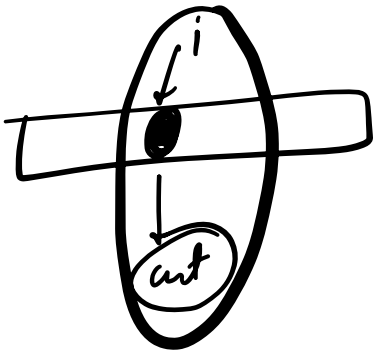
#cnt lesser

I BF

TC = O(N²)

SC = O(1)

```
ANS = 0;
for(i=0 ⟶ N-1){
    cnt = 0;
    for(j=0 ⟶ N-1){
        if(A[j] < A[i]){
            cnt++;
        }
    }
    if(A[i] == cnt){
        ANS++;
    }
}
out ANS;
```



II

A : [1, -5, 3, 5, -10, 4]

SORT ASC

[-10, -5, 1, 3, 4, 5]

idx 0 1 2 3 4 5

A[i] == i

#cnt less

0 1 2 3 4 5

```
// A[]
A.sort();  // ASC   →  N log N
                         +
ANS = 0;                 N
for(i = 0 → N-1){
    if(A[i] == i){
        ANS++;
    }
}
ret ANS;
```

TC = O(N log N)

SC = O(1)

Q SAME AS PREV [elements could repeat]

A : [4, 6, 4, 2, 2, 0]

SORT ASC

[0 2 2 4 4 6]

idx	0	1	2	3	4	5
#cnt lesser	0	1	1 ✗	3	3 ✗	5

A: [-3, 0, 2, 2, 5, 5, 5, 5, 8, 8, 10, 10, 10, 14]

idx: 0 1 2 3 4 5 6 7 8 9 10 11 12 13

#cnt lesser: 0 1 2 2 4 4 4 4 8 8 10 10 10 13

cnt 0 1 2 2 4 4 4 4 8 8 10 10 10 13

```
if( A[i] != A[i-1])
    cnt = i;
else X
```

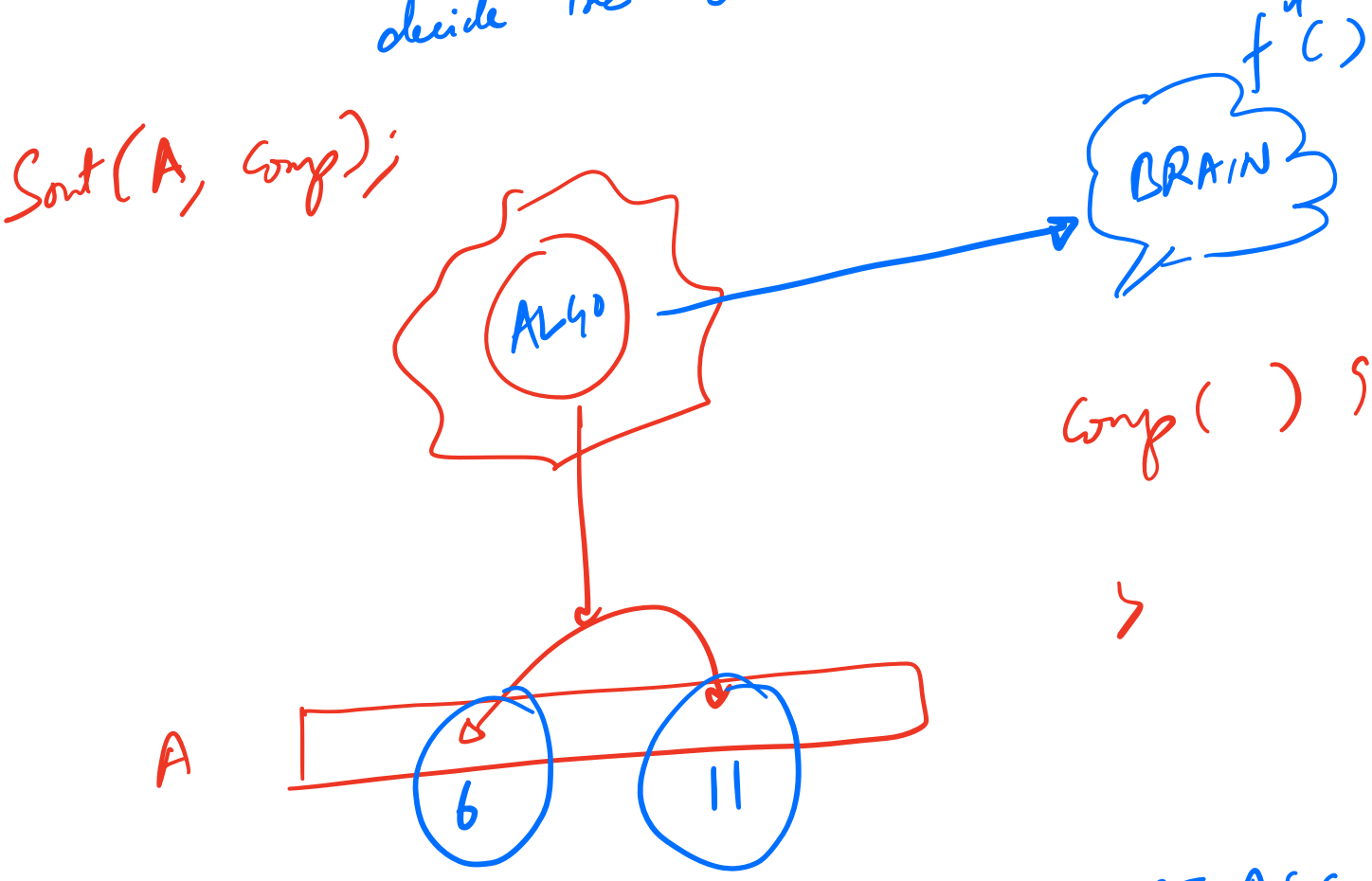
```
ANS = 0;
cnt = 0;
A.sort(); // ASC  → NlogN
for( i = 0 → N-1) {            → O(N)
    if( i==0 || A[i] != A[i-1]) {
        cnt = i;
    }
    if( A[i] == cnt) {
        ANS++;
    }
}
ret ANS;
```

TC = O(NlogN)

SC = O(1)

⊙ Custom Comparators

→ It will help your sorting Algo to decide the order!




```
bool comp(int a, int b) {          → SORT ASC
// Return true if a to be placed before b
// ——— false otherwise
    if(a < b) {
        ret true;
    }
    ret false;
}
```

SORT DESC

```
if(a > b) ret true;
ret false;
```

Sort(A, comp);

Q Sort the Arrq in DEC order of No of factors.
if there is a tie, the smaller should come first!

A: [1, 3, 6, 2, 7, 9] I/P

#fac : [1, 2, 4, 2, 2, 3]

[6, 9, 2, 3, 7, 1] O/P

```
int fact(int n){
    {O} → returns the # of factors of n
}
bool comp (int a, int b){
    int fa = fact(a);
    int fb = fact(b);
    if((fa > fb) || ((fa == fb) && (a < b)))
        ret true;
    ret false;
}
Sort(A, comp);
```

$\sqrt{MAX(A)}$

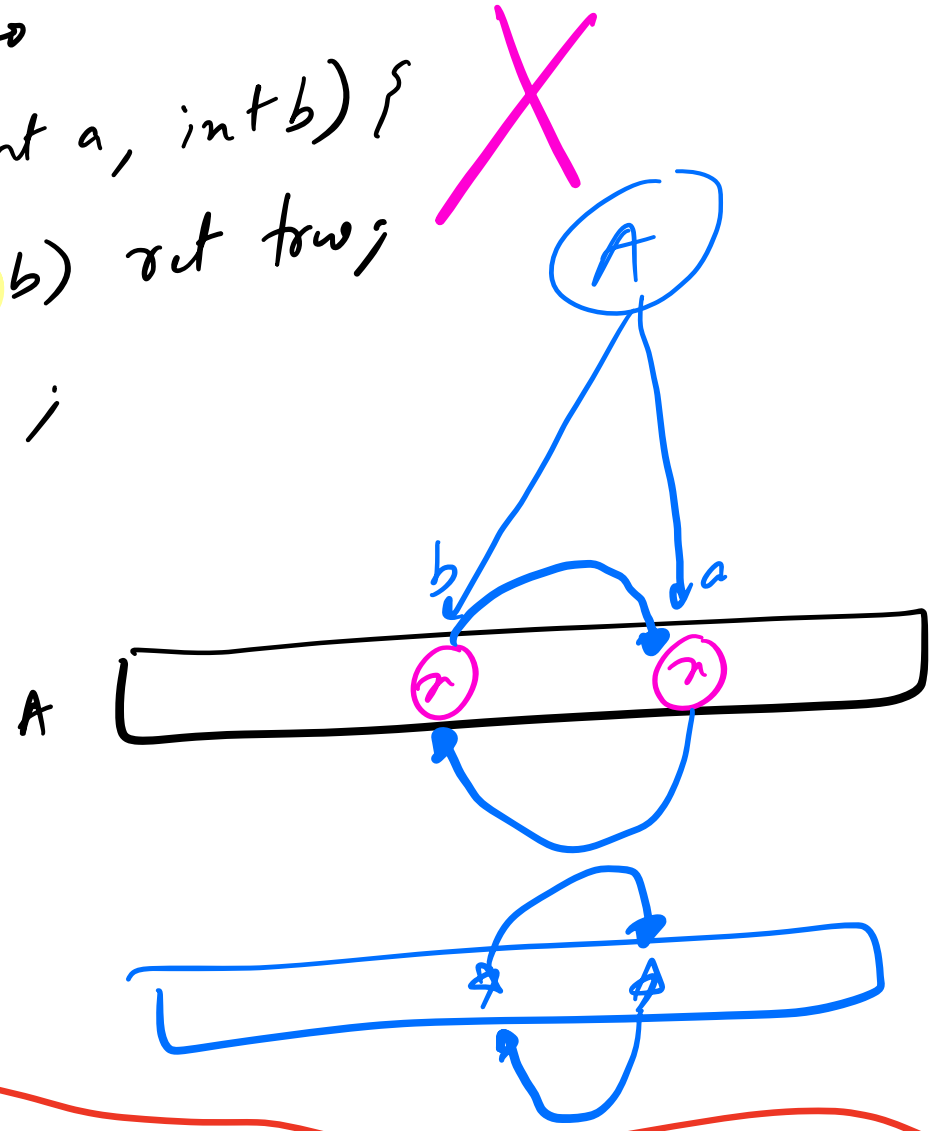
$\sqrt{a}$

$\sqrt{b}$

$TC = O(\sqrt{MAX(A)} \times N \log N)$

SORT ASC

```
bool comp ( int a, int b) {
    if ( a <= b) ret true;
     ret false;
}
```



NOTE :

- ALWAYS RETURN FALSE for SAME PRIORITY Elements